MultiMod II Bootloader

A Forth CPU Based Customizable Bootloader

Mark A Fleming

Table of contents

1. Home

1.1 Forth Bootloader for the MultiMod II

The J1a soft Forth CPU was selected in order to implement the intelligent functionality of the MultiMod II as an HP-71B accessory. The use of Forth as the base language complemented the availability of Forth for the 71B itself.

The mecrisp-ice project combines the J1a Forth CPU with the Mecrisp ANSI Forth implementation that targets the J1a instruction set. Details about the J1a implementation can be found at James Bowman's github repository. More information about Mecrisp Forth can be found at its Sourceforge site. Unofficial documentation for Mecrisp Forth can be found on Sourceforge as well.

This version of the J1a its functionality with the hardware needed to perform USB detection and warm boot of a selected bitstream. The warm boot capability will also be present in the default working bitstream configuration so that the MultiMod II can be rebooted into a different configuration of the owners choice.

1.1.1 Bootloader Purpose

An FPGA will load its design configuration, stored as a bitstream in external flash memory, when it first powers up or is forced to reset. Putting this configuration in flash requires equipment that is likely unavailable to a MultiMod II owner. A bootloader is a design configuration that will sense whether the MultiMod II is connected to a USB host. If so, it allows the host to update a working design configuration and then transfer control to that "production" configuration. If not connected to a USB host, control is immediately transferred to the working production configuration.

There are a number of available bootloaders for the Lattice iCE40 FPGAs. A bootloader provides the means of updating the working FPGA bitstream configuration without the need for equipment to update the flash memory storing the configuration. Most bootloaders are a simple affair, providing only the means of updating the working FPGA bitstream configuration. However, the MultiMod II also requires a means of transferring user data files between the MultiMod II flash memory and the user PC.

The desired interactivity with the MultiMod II is complex enough to use the J1A Forth CPU in place of a simple single-purpose bootloader. This also means that the bootloader program itself can be augmented by the MultiMod II owner to extend its capability even after its purchase.

1.1.2 Project Layout

Board design and documentation for the mecrisp Forth implementation can be found in separate directories beneath the main MultiMod-II directory. The `mecrisp-ice-x.xx` directory contains only the board support directories for the MultiMod II. The `mmj1boot` directory contains the necessary build scripts, the default Forth files that support MultiMod II operation, and the board definition files. See the Build section of this documentation for details.

1.1.3 Forth Operation

The current mecrisp-ice configuration supports the default implementation that communicates using the USB device for the default user console. An initial Forth dictionary that is a part of the bootloader configuration bitstream contains Forth words needed to support desired interactivity. The J1A hardware design has also been augmented with the necessary warm boot design components so that control can be transferred by loading an alternative FPGA bitstream configuration such as the production bitstream design to interface the MultiMod II with the HP-71B bus. Details are provided in the appropriate sections of these Notes.

1.2 MultiMod II for the Hewlett Packard HP-71B Pocket Computer

The MultiMod II is a successor to the MultiMod ROM emulator board for the HP71B. The original board fit into the card reader slot of the HP-71B and could emulate up to 120 KB of ROMs.

The MultiMod II can emulate up to 128 KB of mixed ROM, RAM, and Independent RAM (IRAM). ROM and IRAM images are stored in a 16 MB flash memory device. Images can be loaded, unloaded, or saved by user command. The HP-71B owner can configure ROM and RAM on the fly to match current needs, and backup IRAM content to flash so that no work is lost if batteries fail.

1.2.1 Design Decision Issues

There are several design decisions that affect the functionality and usability of the MultiMod II. First and formost is the choice of a bootloader. The are a number of suitable bootloaders for the Lattice FPGA platform, but they are limited to making production updates, with little or no support for other operations.

The choice of how to implement a file system in flash for Forth dictionary images and for HP-71B ROM/IRAM images has a number of constraints that must be managed. Another issue is whether to have separate directories for Forth images and HP-71B images, or just a single combined directory.

The user interface for the MultiMod II when plugged into a USB host has important implications for ease of use by owners of differing technical sophistication. The interface should make software updates simple, while also supporting files transfers with a minimal host-side client requirement.

1.2.2 Future Enhancements

The MultiMod II is designed to easily update the FPGA configuration and Forth support software via its USB connection. Plans also include support for the embedded Forth machine to access IRAM file systems when installed in an HP-71B and to act as a coprocessor for the 71B itself.

The Forth software and modified Verilog for the MultiMod II adheres to the overall license for James Bowman's mecrisp-ice (BSD 3-Clause "New" or "Revised" License) distribution.

2. Build Instructions

2.1 MultiMod II Bootloader Build Instructions

These instructions assume a Linux build environment, specifically Pop!_OS. Other Linux distributions are likely sufficient if they are supported by the toolchains used herein. The author has used WSL (Windows System for Linux) as well for early exploration.

2.1.1 Setup

This version begins by building the design in the `mmj1boot` directory using the Fomu development platform's designer-provided toolset. The tar file containing the icestorm tools should be installed in /opt/fomu-toolchain-Linux. Place its bin subdirectory FIRST in your PATH environment variable.

```
export PATH=/opt/fomu-toolchain-Linux/bin:$PATH
```

Use `sudo apt install gforth` to install an old version of gforth in order to compile the latest Gforth source files. The older default version of gforth works fine, though I did build the latest version from source just in case any significant changes or improvements to gforth occurred.

2.1.2 Compilation and Synthesis

Within the `mmj1boot` directory use the `compile` command script to create an initial Forth dictionary image using `compilenucleus` and using the above icestorm toolset to synthesize the J1A CPU design. An unrecognized option appears in the first command in the compile script that invokes yosys, the `-noabc9` option. The build completes successfully with this option removed.

The script needs a seed value when invoking the `nextpnr-ice40` place and route program. The value used (11) was determined by successively trying values, starting with 1, until a seed was found that allowed synthesis to complete with all clock constraints satisfied. At the end of the build the bitstream file `j1a.dfu` will appear in the `mmj1boot` directory.

Within the *build* directory will be found the `j1a.bin` and `multiboot.bin` files. These binary files can be used to program flash memory on the MultiMod II board. The `multiboot.bin` file incorporates both this bootloader bitstream and the production bitstream configuration as the second image.

2.1.3 Development Testbed

Developing the Forth code to support interaction with the FPGA and the external SPI flash memory can be done with a development environment that replicates some of the MultiMod II physical board. One such testbed is the Fomu development board that fits in a USB slot.

Using the **Fomu** testbed:

Run `./compile` in the `fomu` subdirectory to build a new bitstream image.

Run `sudo /opt/fomu-toolchain-Linux/bin/dfu-util -l` to list the attached DFU device(s) identification status.

Run `sudo /opt/fomu-toolchain-Linux/bin/dfu-util -D j1a.dfu` in the fomu subdirectory to load the newly built bitstream image. You may need to supply the vendor and device id as parameters to the command if more than one DFU capable device is attached. I recommend having only the Fomu attached to avoid confusion or mistake.

Run `sudo /opt/fomu-toolchain-Linux/bin/dfu-util -e` in the fomu subdirectory.

Run `sudo dmesg | grep tty` to identify the tty to which Fomu is attached. By default it should be /dev/ttyACM0```

Communication

Use `minicom -b 115200 -D /dev/ttyACM0` to connect to the Forth CPU. Note that you'll need to add yourself to the dialout group. An alternative terminal program is GTKTerm, which is more easily configurable.

Invoke minicom communication (control-A Z) then set lineWrap on/off (control-A Z W) followed by Add Carriage Ret (control-A Z U). Set the comm parameters to 8N2 (control-A Z P X). You'll need to add some delay to each character sent by minicom to the Forth console via the terminal settings (control-A Z T). Set the character send delay to 1 milliseconds and the line end delay to 50 milliseconds. **The same character delay will be needed by other terminal emulators, such as TeraTerm.**

For the above settings to work properly, use the `dint` command at the start of your working session to disable interrupts.

Note: to avoid using sudo each time you run *dfu-util*, you can change the permissions for the usb device. First, identify the device using the lsusb command

```
lsusb Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub Bus 003 Device 113: ID 1209:5bf0 Generic Fomu PVT running DFU Bootloader v2.0.3 Bus 0
```

The device is identified as **Generic Fomu PVT running DFU Bootloader v2.0.3**. Note the Bus and Device numbers. For the above example, go to /dev/bus/usb/003, then do a chmod 0777 113 to give permissions to all. From there, you can run *dfu_util* as a regular user.

2.1.4 Production Build

To build a bitstream image for the MultiMod II board itself, change to the `mmj1boot` subdirectory then execute the *./compile* script. Use the `build/multiboot.bin` file to program the MultiMod II flash memory.

MultiMod II Board Programming With The CH341A

First, if not already present, install IMSProg using the commands

```
sudo add-apt-repository ppa:bigmdm/imsprog sudo apt update sudo apt install imsprog
```

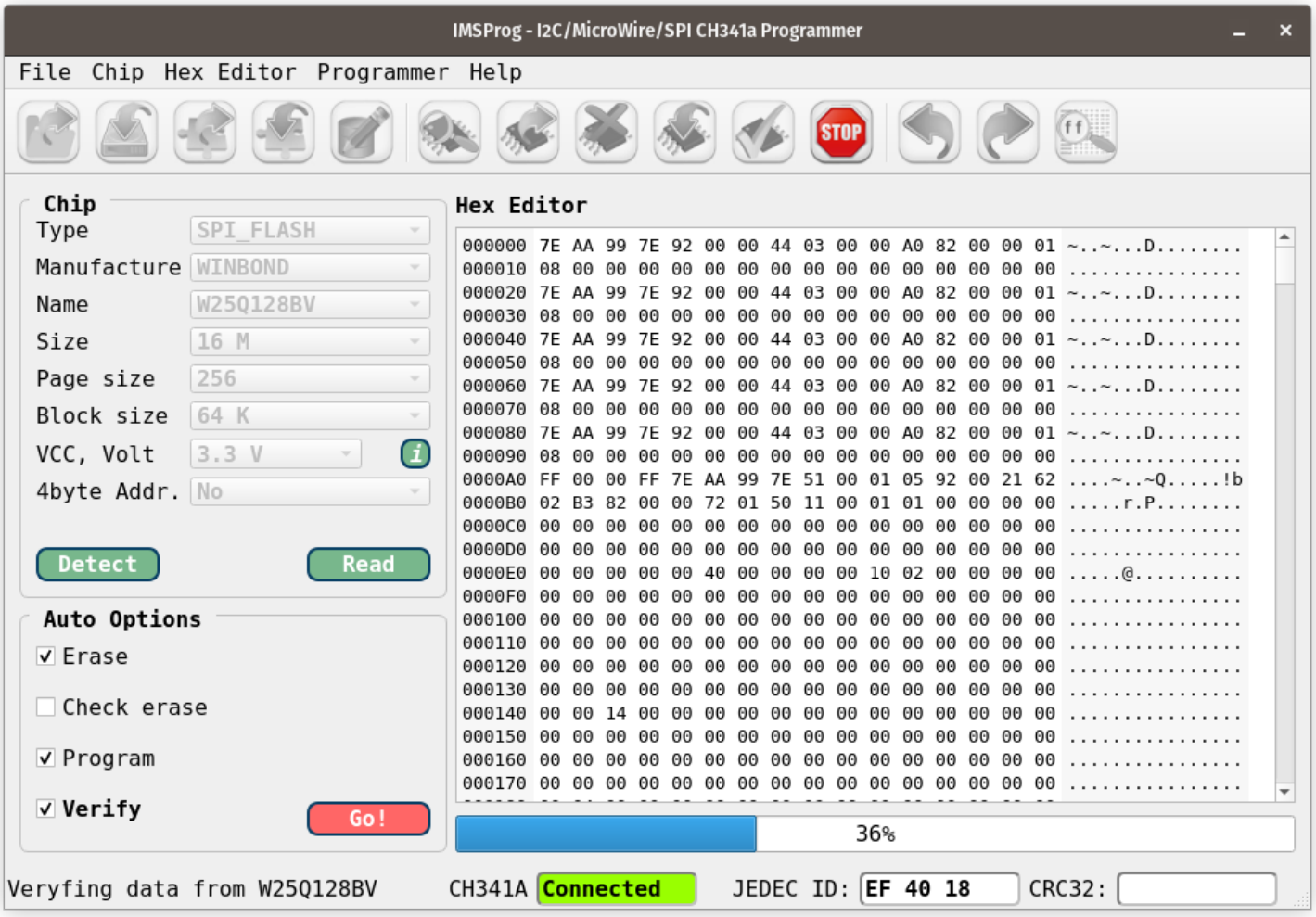
Using the programming adapter board, connect the header cable to the MultiMod II header and to the programming adapter board. The red orientation stripe should be at the top of the MultiMod II header and at the top of the programming board header. Put the pins of the adapter board in the lower half of the CH431A programmer socket, then plug the programmer into a USB socket. The programmer presence can be verified with the `lsusb` command and should appear similarly to that below.

```
Bus 003 Device 053: ID 1a86:5512 Qin Heng Electronics CH341 in EPP/MEM/I2C mode, EPP/I2C adapter
```

The following command will bring up the program utility GUI (be sure you are in the `dialout` group)

```
IMSProg
```

Pressing the `Detect` button should show the Winbond 16 MB SPI flash device with JEDEC ID EF 40 18. Use the Open button to read in the `multiboot.bin` file, then press Go to program the bootloader into flash.



3. Build Files

3.1 The J1A Forth CPU Verilog File

3.1.1 iCE40UP5K FPGA Pin Assignments

The various MultiMod II signal connections are given in the table below. Signals are grouped by function rather than IO block/pin number.

HP-71B Bus Signals

```
Signal FPGA Name SG48 Pin ------------ ----------------- ------------ Data IO 0 IOB_0a 46 Data IO 1 IOB_2a 47 Data IO 2 IOB_5b 45 Data IO 3 IOB_3b_G6 44 S
```

HP-71B Bus Level Shifter Control Signals

```
Signal FPGA Name SG48 Pin ------------ ----------------- ------------ Data Out Enb IOB_4a 48 Data In Enb IOT_51a 42 Cntrl In Enb IOB_50b 38
```

External Device Control Signals

```
Signal FPGA Name SG48 Pin ------------ ----------------- ------------ USB Pos IOT_38b 27 USB Neg IOT39a 26 USB Active IOB_6a 2 USB Pullup IOT_42b 31 Osc E
```

External Flash Storage Signals

```
Signal FPGA Name SG48 Pin Flash Pin ------------ ----------------- ------------ ----------- ----- SPI_SO IOB_32a 14 DI (IO0) 5 SPI_SI IOB_33b 17 DO (IO1)
```

3.1.2 CPU External Signal Modifications

The J1A module definition was modified to include an output pin for the external clock oscillator enable signal. The `clk_en` signal should be set to logic high by default.

The pins associated with the USB data signals and pullup are `usb_dn`, `usb_dp`, and `usb_pu`. These signals are mapped to FPGA pins not enumerated in the default J1a CPU.

Likewise, the `to71`, `from71`, and `ctrl71` outputs that control the tristate level shifters are added to the default J1a CPU interface. These also should be set to logic low so that the buffers are tri-stated. These may not be necessary since the enable inputs are pulled low and unconfigured IO pins are likely left floating.

Check this in the FPGA data sheet

The header is

```
module top ( input clki, // 48 MHz clock input output clk_en, // Enable external clock inout pmod_1, // Four user pins inout pmod_2, inout pmod_3, inout p
```

Note that the four user pins `pmod_(1,2,3,4)` were part of the Fomu board definition and are mapped to bus data I/O pins on the MultiMod II board. These signals should be renamed prior to formal release of the Verilog design source.

The three RGB pins could be removed for power saving reasons, though doing so would require further modification to the Verilog source to remove references to the signals. The pins are unconnected to any external device.

One set of external signals currently missing from the module header are those that interface to control inputs from the 71B bus. These signals are named and assigned to FPGA pins in the `pcf` file, and eventually need to be incorporated for a J1A Forth implementation operating in production mode within the 71B card reader cavity.

The clock enable output is set within an `always` block, as so

```
always @(posedge clk) begin clk_en <= 1'b1; // Always enabled to71 <= 0'b1; // Always tristated from71 <= 0'b1; // Always tristated ctrl71 <= 0'b1; // Alw
```

3.2 The MultiMod II Pin Definitions

The pin assignments in the `multimod-evt.pcf` file map the signals declared in the module header of `j1a.v` to actual pins in the Lattice FPGA SG48 package.

3.2.1 Pin Assignments

The `pmod_(1,2,3,4)` names match those declared in the j1a.v module declaration and will need to be renamed in both files to something like the `Bio(0,1,2,3)` names found on the MultiMod II schematic.

```
set_io rgb0 39 set_io rgb1 40 set_io rgb2 41 set_io pmod_1 46 set_io pmod_2 47 set_io pmod_3 45 set_io pmod_4 44 set_io data_0 46 set_io data_1 47 set_io
```

3.2.2 Trivia

In the original MultiMod II design the pins for `clki` and `clki_en` were swapped. This worked fine for the mecrisp-ice design and so four test boards were ordered. It quickly turned out after receiving the test boards that the two candidate bootloader designs could not be synthesized using this pin assignment, and so the pins were swapped, new boards were orderd, and the old test boards were scrapped.

4. Functionality

4.1 Features and Requirements for the MultiMod II Bootloader

As an accessory for the HP-71B, the MultiMod II is meant to provide both ROM and RAM plugin modules. All known ROMs should be available for access by a MultiMod II owner. The owner should also be able to plug in RAM modules, either in merged form or as independent RAM (IRAM). The content of IRAM modules should also be able to be saved to flash memory so its content can be accessed at a later time.

In addition to providing regular relocatable ROM and RAM modules, the MultiMod II will also support fixed memory at two locations; a fixed hard ROM at location $E0000 and a takeover ROM at location $00000. The former is used by the Forth/Assembler ROM among others. The latter requires a separate shorting module in Port 1.

4.1.1 Bootloader

The Forth bootloader serves three purposes; to act as a bootloader that updates the MultiMod II production configuration under user control, to transfer files between the MultiMod II and a user host PC, and to develop Forth applications for use by the production Forth CPU in the HP-71B.

First, a hardware modification to the mecrisp-ice J1a CPU is needed to support warm boot. The modification allows a Forth program to initiate the warm boot process by supplying the address of a new bitstream pattern in flash and initiating the warm boot design confirguration replacement process.

Second, a user interface is needed for updating the FPGA production configuration and transferring files between the MultiMod II and the user host. Given the USB-based Forth console for communication, a terminal application like TeraTerm with a file transfer protocol like Kermit should be a sufficient client for the owner.

4.1.2 File Storage

Transferring files implies a file system, and that means some form of a file identification structure is needed. There are a number of open source flash file systems, but these assume small sector sizes (256/512 bytes) and plenty of host RAM. Neither condition is true for the MultiMod II where flash sector size is 4KB and only a fraction of the 15KB Forth dictionary is available for scratch space.

The 16MB flash memory will be treated as a structure of 1024 blocks of 16KB each, where 16KB is the size of a Forth dictionary image or a size multiple of HP-71B ROM and RAM images.

Transfer of ROM image data in the MultiMod was by sending ASCII Hex representation of ROM data via a serial port. While this mode is still supported by the MultiMod II, the Kermit protocol will be used to transfer raw binary image files between the MultiMod II flash and a host PC. This protocol is also suitable for transferring updates to the production FPGA bitstream image.

4.1.3 Forth Development

There is little reason to develop Forth applications on the HP-71B itself when one can interact with the USB-based Forth console from a host using a terminal client. Furthermore, adding words to the Forth dictionary is a simple matter of sending a text file containing Forth definitions via the client. Thus, text development can be performed on the host, and a target dictionary can be saved to flash for later use in the HP-71B production environment.

4.1.4 Optional: HP-71B Emulation

If an owner wishes to develop a Forth application that can interact with the HP-71B in actual production operation, then some means is needed to simulate the 71B behaviour. One example would be to develop words that can interact with and modify an IRAM filesystem. A Forth application could then read and write data to local files in their FPGA SPRAM environment.

4.2 MultiMod II Flash Memory

The MultiMod II board has a 16MB Winbond flash device. When the FPGA exits power-up or reset, it reads its initial configuration bitstream from flash starting at address 0. It can optionally read alternative configuration bitstreams from other locations in flash, a process known as **warm boot**.

Flash memory is a byte-serial device rather than random access memory. A flash memory location must first be erased (all one's) before it can be written. The smallest block of flash that can be erased is 4 kilobytes. A read or write operation is always preceded by a command byte followed by a three byte (24-bit) memory address. Data can then be streamed to or from the flash device on a bit serial basis. The design of the MultiMod II allows an optional quad serial transfer interface.

The MultiMod II treats its 16 MB flash memory as a array of 1024 blocks of 16 KB size, with each block on a 16 KB address boundary. The flash memory is divided into three sections; section one for FPGA bitstreams and their associated data, section two for Forth dictionary images and data files, and the remaining third section for 71B ROM and IRAM images. ROM and IRAM images are 1, 2, or 4 blocks in size (16/32/64 KB).

Need a picture here of flash memory and sector addressing

4.2.1 Boot Section

The first megabyte of flash memory is reserved for configuration bitstreams as well as any data associated with a given FPGA configuration. In the **foboot** bootloader, the starting address of a bitstream varies according to post-boot data requirements, but roughly $020000 (128KB) is reserved for each bitstream. For the MultiMod II, four bitstream areas will be reserved in the lower half of the boot section, with the upper half serving as optional data storage for a given bitstream device.

4.2.2 Forth Section

The J1a Forth CPU has its dictionary in the 15KB dual port RAM of the Lattice FPGA. There are **load** and **save** commands which allow the dictionary to be written to flash or read from flash in its entirety. This gives a programmer the ability to develop applications and save them to flash for later use.

The second megabyte of flash memory is reserved for Forth dictionary images. Since an image occupies a 16KB block of flash, the Forth Section has room for 64 dictionary images. The first two blocks are reserved for a directory of images and a scratch block for image manipulation. The remaining 62 blocks store dictionary images.

4.2.3 HP-71B Section

The remaining 14MB of flash is reserved for ROM images and IRAM images. These images are multiples of 16KB in size, hence like the Forth Section storage is divided into 768 blocks. The first two of these blocks are reserved for a directory of images and a scratch block for image manipulation.

5. File System

5.1 MultiMod II Flash File System

Although a MultiMod II owner developing Forth applications might be able to keep track of in which block number a dictionary is stored, the task is far more difficult for the many HP-71B ROM/IRAM images stored in flash. Each image varies in size (16KB, 32KB, 64KB), access (Read-Only, Read-Write), and type (Forth, ROM, hard ROM, IRAM). There may even be different versions of ROMs, as with the MATH ROM.

Thus a file system is needed to support creating, deleting, or renaming a storage entity. In the case of both Forth dictionaries and HP-71B memories, the supported operations are as follows

- **New -** For the HP-71B a new IRAM can be created by giving a name and size.
- **Load -** A named image is loaded into memory.
- **Save -** A Forth dictionary image or a HP-71B IRAM image in memory can be saved to flash under its existing name (Save), or can be saved with a new name (Save As) so as not to overwrite the old image.

5.1.1 Directory Structure

A directory consists of 1024 directory entries in a 16KB flash block. The structure of an individual entry is defined in the next section. Since new entries can only be written to previously erased sections of the flash block, new or modified entries are written after the last entry in the directory list.

Entries grow upward within the directory block, with deleted entries marked as reclaimable. When updating an existing HP771B ROM/IRAM image, the size of the image remains the same as when the image directory entry was first created. That is, the resizing of an image is not supported.

5.1.2 Directory Entry

A directory entry is sixteen bytes in length, with twelve bytes devoted to the entry object name, two bytes to the image starting block number, and two bytes devoted to the attributes of the object. To support directory relocatability, the image starting block number is a offset from the beginning block of the directory itself.

Flash bytes are all ones when erased, so writing to flash consists of setting ones to zeros in a given byte. If bits are considered flags with default value set to one, then a bit in one of the attribute bytes can be forced to zero to indicate the entry is no longer valid.

5.1.3 Directory Operations

As mutable images are updated and stored to flash, the most recent entry for an image is located at the upper end of active, unerased flash. Thus directory search starts at the most recently written directory entry, working downward towards the beginning of a directory block.

Directory entries and images both grow upwards within the flash storage allocated to them. Entries and/or images eventually reach the top of their allocated storage area, and so entries and images that are no longer valid must be eliminated in a process called *packing*.

5.1.4 Directory/Storage Packing

The packing process begins by scanning upwards in the directory searching for reclaimable entries. For each such entry, the 16KB blocks making up the image that the directory entry points to are erased. This erased space will later be occupied by valid images as part of the packing process.

Directory packing involves copying valid directory entries over to an erased scratch block the same size as the directory block. As directory entries are copied, their corresponding images they point to are also moved downward in flash address space.

Downward movement of images involves keeping track of the lowest erased block, then copying the image one block at a time. Each image block copied downward is erased after the copy completes. This step is necessary not only to provide room for images above the current image, but also to handle the case when there are not enough empty blocks below the image to hold the complete image. Erasing blocks as they are copied provides the needed space for subsequent image blocks.

5.2 Directory Implementation

The following provides implementation detail for an image directory. As per the previous section, a directory occupies a single 16KB block of flash memory, and is divided into 1024 directory entries of 16 bytes each. A directory is thus able to addess the entirety of flash memory, which itself is divided into 1024 images blocks of 16KB each.

Multiple directories and their associated image blocks can exist in flash. The initial implementation will have separate directories for Forth dictionary images and HP-71B ROM/IRAM images. The bulk of flash storage will be allocated to HP-71B images, which can be up to 14 MB. The base implementation will have only these two directories.

5.2.1 Directory Structure

A directory is a collection of 16KB image blocks in flash. The first 16KB block is the directory itself, followed by a scratch 16KB block used by the packing process. Following these two 16KB blocks are a fixed number of 16KB image blocks associated with the directory. This size value is encoded in the first directory entry of the directory itself. Each directory collection can be packed separately.

5.2.2 Directory Entry

A directory entry specifies an image name, type, size, and location. Starting with the first entry byte, the entry structure is

- (0) **Entry Status.**
  This byte indicates the entry status, whether it is **Empty** ($FF), **Valid** ($F0), or **Reclaimable** ($00).
- (1) **Image Type.**
  An image type is encoded in the upper four bits of this byte and its size is encoded in the lower four bits. The image size is the number of 16KB image blocks given by the lower four bits plus one.
  The image type is **IRAM** ($0x), **ROM** ($1x), **HARD ROM** ($2x), **ROM+HARD ROM** ($3x), and **TAKEOVER ROM** ($4x). Note that the size of a HARD ROM and TAKEOVER ROM are fixed, so that the size bits are ignored. The size bits of a ROM+HARD ROM encode only the size of the ROM itself.
  As an example, the Forth/Assembly ROM combined with its companion Hard ROM would be encoded as $20 (ROM+HARD ROM, size one 16KB image block).
  The first entry of a directory is of type **SIZE** ($8x). The lower four bits are ignored, and the Image Location field indicates the number of image blocks that are inclusive to the directory.
- (2~3) **Image Location.**
  The location of images could be encoded as an absolute position within flash memory, or as a offset from the beginning of the directory.
- (4~15) **Name.**
  An image name can be up to 12 ASCII characters, padded with $FF bytes when less than 12 characters.

6. Forth System

6.1 The j1a Forth CPU

The original j1a CPU was developed by James Bowman. The Verilog source can be found at the following github repository.

The version used in the MultiMod II was used to port the Mecrisp Forth implementation, originally developed for Stellaris ARM microprocessors, to the Lattice iCE40 family of FPGAs.

6.1.1 MultiMod II CPU Origins

The j1a CPU that is part of the MultiMod II is the Mecrisp-Ice for FPGA source found on Sourceforge. The Forth language implementation originated with James Bowman's *Swapforth*, found alongside his j1a CPU implementation. From the source description

Mecrisp-Ice is a 16 bit Forth running on a stack machine specifically developed for FPGAs, originally based on Swapforth and the J1a processor by James Bowman. Mecrisp-Ice requires initialised single-cycle dualport RAM blocks to run and is developed with excellent realtime capabilities and deterministic interrupt timing in mind. Due to instruction set design, the maximum (and recommended) amount of addressable executable memory is 16 kb, with an usable minimum of 8 kb.

The 16 bit implementation is stable and rock solid, whereas the 32 bit and 64 bit implementations with support for larger executable memories should be considered experimental.

6.1.2 MultiMod II CPU Modifications

The following modifications have been made to the stock Mecrisp-Ice j1a CPU implementation.

CPU Warm Boot Support

The iCE40 UltraPlus, like all iCE40 FPGA's, supports warm boot. Only the HX/LX series supports cold boot. The warm boot feature allows a configured iCE40 FPGA to dynamically reconfigure itself from one of four bitstreams within an attached serial flash device. The

default numeric limit itself can be easily extended to many more bitstream configurations. The reconfiguration process is mediated by the so-called *boot applet* stored at the beginning of flash memory.

Within the IceStorm toolset (available for both Linux and Windows WSL) the `icemulti` command is used to concatenate the boot applet and up to four bitstream images generated separately by the IceStorm toolset. Command options determine such things as which bitstream is loaded on cold boot and where images are stored in flash.

For a bitstream image to initiate a warm boot, the **SB_WARMBOOT** IP instance must be incorported into the Verilog design. This instance has two inputs that select which of four bitstream images to load, and one input that serves to trigger the warm boot process. In the MultiMod II bootloader, these three signals are controlled by the j1a Forth CPU and the process is under program control.

```
// ###### Warm Boot Control ############################## // Add control register to j1a address space reg [2:0] BOOTCTL = 0; SB_WARMBOOT B_WARMBOOT( .B
```

SRAM instantiation

Information on modifications for HP-71B bus access.

6.1.3 MultiMod II CPU Address Space

The j1a CPU selected for the MultiMod II is a 16-bit word size, 16-bit address size processor. The memory address space for stack and dictionary is mapped to the 15KB of dual-port RAM blocks in the FPGA. Access to this memory is on a single cycle clock basis.

The processor also addresses a dual external address space arrangement; a 64K word RAM address space, and a 64K word I/O address space. The FPGA's 128KB of single port memory, organized as four 16KB by 16 bit RAM blocks, occupies the entirety of the RAM address space.

The I/O address space maps device registers, with the default for the Mecrisp-Ice design shown below.

```
I/O Address Device ----------- ------------------------- 0008h LED register (R/W) 0009h LED register AND data (W) 000Ah LED register OR data (W) 000Bh LE
```

Note above that the last two table entries indicate that any I/O address where bit 12 is set will address the Forth console data register (USB or serial). Likewise, if bit 13 is set, the terminal status register (data valid, data ready).

The **Boot Control Register** at I/O address 00A0h has been added to the address map to control the warm boot process. The three bit register uses the lower two bits [1:0] to select which of four configuration bitstream images to select while the high order bit [2] will trigger a warm reboot when set to logic 1.

6.2 Mecrisp Forth Implementation

The mecrisp-ice project combines the J1a Forth CPU with the Mecrisp ANSI Forth implementation that targets the J1a instruction set. Details about the J1a implementation can be found at James Bowman's github repository. More information about Mecrisp Forth can be found at its Sourceforge site. Unofficial documentation for Mecrisp Forth can be found on Sourceforge as well.

From the mecrisp Sourceforge page on **Mecrisp-ice**:

Mecrisp-Ice is a 16 bit Forth running on a stack machine specifically developed for FPGAs, originally based on Swapforth and the J1a processor by James Bowman. Mecrisp-Ice requires initialised single-cycle dualport RAM blocks to run and is developed with excellent realtime capabilities and deterministic interrupt timing in mind. Due to instruction set design, the maximum (and recommended) amount of addressable executable memory is 16 kb, with an usable minimum of 8 kb.

The 16 bit implementation is stable and rock solid, whereas the 32 bit and 64 bit implementations with support for larger executable memories should be considered experimental.

7. Forth Files

7.1 Acknowledgements

Select Forth code in this file is derived from UPduino-Mecrisp-Ice-15kB by Igor-m BSD 3-Clause license

See: https://github.com/igor-m/UPduino-Mecrisp-Ice-15kB.git

7.2 Flash Memory Structure

Flash memory can't be written on an individual memory cell basis; a sector must be erased and then the modified contents of the sector written back. Data can be written in smaller amounts to a sector but the sector must first be erased. In the Winbond standard SPI flash device, sectors can be erased in 4 KB, 32 KB or 64 KB sized blocks.

The MultiMod II treats its 16 MB flash memory as a array of 1024 blocks of 16 KB size, with each block on a 16 KB address boundary. The flash memory is divided into three sections; section one for FPGA bitstreams and their associated data, section two for Forth dictionary images and data files, and the third section for 71B ROM and IRAM images. ROM and IRAM images are 1, 2, or 4 blocks in size (16/32/64 KB).

7.3 SPI Flash Memory Support

The words in this file provide support for SPI flash memory devices using a "bit-bang" approach to communication. This admittedly simple approach to implementing an SPI interface is nonetheless fast enough for purposes of the MultiMod II project.

The Low Level Support section describes the software implementation of the SPI hardware protocol. Should the built-in hard SPI fabric be used (or soft SPI driver), its code would replace the words found in the Low Level Support section.

The API Functions section are words used by the remainder of the MultiMod II support code. These functions treat the SPI flash memory as a sequence of 16 KB pages - in the case of a 16 MB flash device, pages 0 to 1023.

7.3.1 Constants and Variables

The following definitions define boundaries within flash for bitstream, Forth dictionaries, and ROM/IRAM images. Low level words use these values to partition flash and protect critical sections.

- **bitfence (constant 8) -** Bitstream Partition Point.
  The 16KB page below which no writes can occur. Flash address $20000 or 128KB.
- **frtstart (constant 64) -** Start Of Forth Dictionary Images.
  Starting 16KB page where Forth dictionary image storage begins. The first two pages are reserved for the image directory. Flash address $100000 or 1MB.
- **romstart (constant 128) -** Start Of HP-71B ROM/IRAM Images.
  Starting 16KB page where HP-71B image storage begins. The first two pages are reserved for the image directory. Flash address $200000 or 2MB.

7.3.2 Low Level Support

The Forth words in this section implement the software-based SPI interface. These functions are not used by ordinary user functions.

- **idle (--)** - Disable Device.
  Deasserting the Chip Select pin will put the flash memory chip in a quiescent power mode. But it is also needed for other commands, such as the sector erase commands where the chip must be deselected immediately after sending the command.
- **spixbit (x -- y)** - Read/Write Data Bit.
  The high order bit of the 16-bit word **x** is shifted out to the SPI data out pin and then the SPI data in pin is shifted into the low bit of **x** returning **y**.
- **spix (outdata -- indata)** - Read/Write Data Byte.
  Used follow a Read or Write command. Data on the stack is written to flash following a Write command but is ignored following a Read command. Data returned by this function is valid following a Read command but is an undefined value following a Write command.
- **>spi (-- byte)** - Read Data Byte.
  Use `spix` to return a read data byte (zero if used with a write command).
- **spi> (byte --)** - Write Data Byte.
  A zero byte is shifted out while the read data is shifted in and the data byte is returned on the stack.
- **waitspi (--)** - Wait For Device Ready.
  Checking for the Write In Progress flag to go low is needed immediately after sending an Erase Sector command.
- **spiwe (--)** - Enable Writing To Device.
  On application of power, the flash device will not allow erase or write commands until it is unlocked. There is a corresponding disable command. though it is currently not needed. In addition, there are sector write disable commands that could be added to protect the bitstream section of the chip from accidental overwrite.

7.3.3 API Support Functions

The words in this section are used only by the API function words. These functions do not implement any address range restrictions.

- **sect2addr (sector16k -- double_rom_addr)** - Sector Number to 24-bit Address.
  Convert a 10-bit sector address word (0~1023) into a 24-bit double-word address suitable for an SPI Read or Write command.
- **addr2sect (double_rom_addr -- sector16k)** 24-bit Address to Sector Number.
  Convert a double-word containing a 24-bit flash address to a sector number. The lower 14 bits are truncated.
- **addr2spi (double_rom_addr --)** Output 24-bit Address to SPI.
  The 24-bit address in a double-word is written as three bytes to the SPI interface, high byte first. Bytes are written high bit first.
- **spiread (double_rom_addr --)** - Address A General Memory Location.
  This function can be used to read from flash memory starting at any location. Subsequent >spi commands will return each sequential data byte after a first dead read cycle.
- **spiread16k (sector16k --)** - Address Start Of Memory Sector.
  This function is passed a 16 KB sector number in the range of 0 to 1023 for the 16 MB SPI flash memory device. Sets up a read operation like the `spiread` command.

7.3.4 SPI Utility Functions

- **spiflush (Nbytes --)** - Read And Discard Memory Bytes.
  This function is provided as a means of advancing the read memory address without having to reissue an spiread/spiread16k command.
- **spidump (Nbytes --)** - Read And Display Memory Bytes.
  This function is provided as a means of examining the read memory address content.
- **numsectors (-- sector16k_count)** - Return Number Of Memory Sectors.
  This function uses the second byte from a Manufacturer ID, returned by the Read JEDEC ID command, to determine flash memory capacity. Useful in place of a hard coded size value.

7.3.5 API Functions

These words, developed for the MultiMod II, are the formal interface to flash memory. User programs can use them to access or update image or data files. These words prevent write access to bitstream data in section one of flash memory. They **do not** limit Forth dictionary images to section two of flash memory.

- **erase4k (sector4k --)** - Erase A 4 KB Sector.
  This command uses a standard JEDEC command to erase the smallest size sector in flash memory. Rather than pass a full 24 bit memory address, the function is given a sector number from 0 to 4095. This insures the address sent in the command is at the beginning of a given sector.
- **erase (sector16k --)** - Erase A 16 KB Page.
  Since the MultiMod II treats flash memory as a sequence of 16 KB blocks, this command is used to erase one such block in preparation for writing. The function takes a block number from 0 to 1023.
- **load (sector16k --)** - Load Forth Dictionary.
  The Forth CPU dictionary is in the 15 KB Dual-Port Memory of the FPGA. This command will overwrite that internal memory with the content of a specified 16 KB block in flash memory. Input is the range 0 to 1023.
- **save (sector16k --)** - Save Forth Dictionary.
  The content of the Forth CPU dictionary is written to a specified 16 KB block location in flash memory. The state of the Forth CPU can thus be restored using this command, allowing different Forth configurations for different tasks.
- **hp71b** - Mark Top of Fixed Dictionary Words.
  The words defined in this and the constants file are the basis of all other code accessing the SPI flash memory. This marks the root of all other versions of the Forth dictionary.

7.4 Lattice FPGA Multiboot Support

The Lattice iCE40 series FPGAs support both warm and cold boot capability in which the FPGA can dynamically reconfigure itself to meet differing external requirements. In the MultiMod II design, this feature is used to configure the FPGA as a bootloader that can update the production configuration bitstream when attached to a USB host, and otherwise configure itself with the production configuration to support its function within the HP-71B.

This Forth module supports a single function used to load a configuration bitstream using the warm boot approach.

- **warmboot (image_num --) -** Boot Into Selected Bitstream Image.
  The *image_num* value is between 0 and 3, where 0 is the default bitstream image loaded duing powerup or reset, and 1 is the default HP-71B production image. When bit 2 is set, the warm boot is initiated using the selected bitstream from bits 0 and 1.

7.5 SPI Flash Security Related Functions

The functionality within this set of Forth words is primarily targeted towards protecting certain areas of flash, i.e. the bootloader, from being overwritten.

- **rdstatreg (reg# -- value) -** Read Status Register.
  Read the content of Status Register 1, 2, or 3. The top of stack is the JEDEC command to read the given status register, either $05 (SR1), $35 (SR2), or $15 (SR3). The return value is the byte value of the given register.
- **wrstatreg (value reg# --) -** Write Status Register.
  Write content to Status Register 1, 2, or 3. The top of stack is the JEDEC command to read the given status register, either $01 (SR1), $31 (SR2), or $11 (SR3).
- **WPSset (--) -** Set Write Protect Status (WPS) Flag.
  The Write Protect Status flag (bit 2 of SR3), when set to 1, allows each 4KB block in flash to be write protected on an individual basis. When the flag is set to 0 then other status registers bits determine write protect protocol. If the flag is set, then all blocks are write protected following power-up or reset, and must be individually write unlocked prior to being programmed.
- **WPSclr (--) -** Clear Write Protect Status (WPS) Flag.
  Clearing the WPS flag is necessary if a previously locked block is to be written.
- **blklock (end start --) -** Lock Blocks In Flash.
  Lock all 4KB blocks in flash from the starting block to the end-1 block. The block range is 0..4095.
- **blkunlock (end start --) -** Unlock Blocks In Flash.
  Unlock all 4KB blocks in flash from the starting block to the end-1 block. The block range is 0..4095.
- **blkstatus (end start --) -** Display Lock Status Of Blocks In Flash.
  Display a series of 1's or 0's for each block in the range start..end-1.

7.6 Flash <--> SPRAM Transfer Support

These Forth words build upon those *soft-spi* low level words for HP-71B ROM/IRAM image interaction with flash storage. The transfers work at the `sector16k` 16KB sector level.

7.6.1 ROM And RAM Addressing

The MultiMod II 16MB flash device is divided into 1024 16KB segments, represented by the numeric value `sector16k`. The HP-71B ROM and IRAM images begin at the `romstart` (2MB) address mark. Valid values for `sector16k` range from 0 to 895.

The 128KB internal FPGA single port RAM is divided into eight corresponding 16KB segments, referred to as `ram#`, whose value ranges from 0 to 7.

7.6.2 Image Transfer From Flash To Internal RAM

The following words copy a ROM or IRAM image from flash to internal FPGA RAM.

- **rom2ram (sector16k ram# --) -** Copy 16KB Image To RAM.
  Copies a 16KB ROM image from flash into a 16KB RAM segment at **ram#**. Valid values are 0 to 7.
- **rom32k2ram (sector16k ram# --) -** Copy 32KB Image To RAM.
  Copies a 32KB ROM image from flash into two 16KB RAM segments, starting with segment **ram#**. Valid values are 0 to 6.
- **rom64k2ram (sector16k ram# --) -** Copy 64KB Image To RAM.
  Copies a 64KB ROM image from flash into four 16KB RAM segments, starting with segment **ram#**. Valid values are 0 to 4.

7.6.3 Image Transfer From Internal RAM To Flash

These flash write commands use the bitstream location restriction feature of the underlying `soft-spi` set of Forth words.

- **ram2rom (ram# sector16k --) -** Copy 16KB Internal RAM To Flash.
  A internal 16KB RAM block **ram#** (usually IRAM) is copied to flash. Flash destination must be erased. Valid values of ram# is 0 to 7.
- **ram32k2rom (ram# sector16k --) -** Copy 32KB Internal RAM To Flash.
  A internal 32KB RAM block **ram#** (usually IRAM) is copied to flash. Flash destination must be erased. Valid values of ram# is 0 to 6.
- **ram64k2rom (ram# sector16k --) -** Copy 64KB Internal RAM To Flash.
  A internal 64KB RAM block **ram#** (usually IRAM) is copied to flash. Flash destination must be erased. Valid values of ram# is 0 to 4.

7.6.4 Internal SPRAM Access

- **ram2ram (ramfrom# ramto# --) -** Copy One 16KB Internal RAM Block To Another.
  This is used to move images stored in internal RAM from one address to another, one 16KB block at a time.
- **ramromcmp (size ram# sector16k -- mem_addr flag) -** Compare SPRAM to flash.
  Compares the image in SPRAM, starting at 16KB page **ram#** (0~7) to the same flash image starting at **sector16k** and is **size** sectors long. The last memory address examined is returned along with a Pass/Fail flag. If the images don't match the **ram_addr** indicates the point of first mismatch.
- **zeroram (ram# --) -** Clear RAM Block To Zero's.
  Clearing a 16K RAM block is done prior to creating an IRAM module. Valid values of **ram#** is 0 to 7.
- **onesram (ram# --) -** Set RAM Block To One's.
  Setting a 16K RAM block to all $FF is done prior to loading a bitstream and writing the content to flash. Valid values of **ram#** is 0 to 7.

7.7 Hex File Transfer Support

The MultiMod transferred ROM images to the IC controller's internal flash memory as a hexidecimal ASCII stream, 32 bytes per line or 64 ASCII characters. This transfer was one-way, to the MultiMod and from the attached host, vis a TTL level serial port. A short Python progrm was used to convert a binary .BIN ROM file to its ASCII equivalent for transfer.

The MultiMod II transfers files between host and device in both directions. The old hex representation method is retained for backward compatibility with existing ROM data files in ASCII format. These Forth words are descibed below.

- **bin2hex (nibble -- char) -** Convert Nibble To Hex Character.
  Convert a 4-bit nibble value to a hex character in the set '0123456789ABCDEF'.
- **hexdump (ram# --) -** Dump 16KB SPRAM Block In Hex.
  Read data from a specified 16KB SPRAM block and output them to the console as a stream of hexidecimal ASCII characters, 64 characters per line. Valid range for `ram#` is 0 to 7.
- **hexdump32 (ram# --) -** Dump Two 16KB SPRAM Blocks In Hex.
  Read data from a specified 16KB SPRAM block and output them to the console as a stream of hexidecimal ASCII characters, 64 characters per line. Valid range of `ram#` is 0 to 6.
- **hexdump64 (ram# --) -** Dump Four 16KB SPRAM Blocks In Hex.
  Read data from a specified 16KB SPRAM block and output them to the console as a stream of hexidecimal ASCII characters, 64 characters per line. Valid values for `ram#` is 0 to 4.
- **hex2bin (-- byte | -1) -** Read Two Hex Characters, Return Value.
  Read two hexidecimal characters from the console and convert them to a byte value. If two carriage returns are encountered in a row, then return a value of -1 to signal the end of the stream of ASCII hex data.
- **hexload (ram# -- nwords) -** Read Hex Stream, Save To SPRAM Block(s).
  Accept a stream of ASCII hex characters from the console, converting them to binary and storing them in a specified 16KB block of SPRAM. Return the number of 16-bit words written to SPRAM. The value should be a multiple of 8 KB words per SPRAM 16KB block.

7.8 Flash File System Directory

A directory consists of two 16KB pages in flash - the first holding the directory itself and the second as a scratch block used when a pack operation occurs. Words relating to creating and maintaining an image directory are defined as follows.

7.8.1 Constants and Variables

These variables are defined for the Forth directory and HP-71B ROM/IRAM directory. They are initialized to the constant values defined in the *ramrom* Forth file.

- **forthdir (variable) -** Forth Directory Location.
  Initialized to the constant value `frtstart`. Use of the variable allows the directory to be relocated if desired.
- **romdir (variable) -** HP-71B Image Directory Location.
  Initialized to the constant value `romstart`. Use of the variable allows the directory to be relocated if desired.

The value of the first byte in a 16-byte directory entry indicates the type of the entry.

- **Empty (constant) -** Unaltered Entry.
  All directory entries except the first are initially empty and their associated bytes are in the erased state.
- **Valid (constant) -** Entry Referencing An Image.
  A directory entry the specifies the name, location, and type of image to which it points.
- **Reclaim (constant) -** Entry Marked For Deletion.
  An entry marked Reclaim has been deleted and its entry space and image are ready to be reclaimed by the packing process.

The second byte of a directory entry indicates both the type and size of the entry's image. The upper nibble of the byte indicates the image type, and the lower nibble indicates the number of 16KB blocks in the image (1~15). The image types are:

- **IRAM, ROM, HARD, TAKEOVER (constants) -** HP71B Image Types.
  ROM and IRAM images are relocatable. A HARD ROM has a starting address of $E0000 while a TAKEOVER ROM begins at address 0.
- **DIRSIZE (constant) -** Internal Use Directory Entry.
  This is the first entry in a directory and indicates how many image blocks follow the two block directory itself. **Note that a future extension could define an entry that points to another directory (subdirectory) within the current directory.**

The third and fourth bytes of a directory entry hold the 10-bit `sector16k` value indicating where the image is located relative to the directory itself. This value is a offset (0~DIRSIZE-1) from the first 16KB block following the directory.

The remaining 12 bytes of the directory entry give the name of the directory image in the form of the string length followed by 1 to 11 ASCII characters.

7.8.2 Directory Support Words

These words support access to and creation/modification of directory entries. Commands that read or write entry values assume the read or write command pointer is positioned to the correct entry and byte offset within the entry.

These words accept the `sector16k` 16KB sector where a directory starts. The words are generalized in order to process either a Forth or HP-71B directory, but can also be used for any user defined directory.

- **writeloc (sector16k --) -** Write Image Block Location.
  The 10-bit `sector16k` value is written to the third anf fourth bytes of a directory entry, low byte first. The value is from zero to the directory size minus one. The 16KB image blocks following the two block directory are numbered starting with zero. The flash Read command pointer must be positioned to the fifth byte of a directory entry.
- **readloc (-- sector16k) -** Read Image Block Location.
  With the flash Read command pointer positioned to the third byte of an entry, this command will read two bytes and return the `sector16k` image location value of the entry.
- **writestr (string --) -** Write Name Of Entry Image.
  Similar to the **writeloc** words, this will write a string length byte followed by string characters. A string longer than 11 characters will be truncated. The flash Write command pointer must be positioned to the fifth byte of the directory entry.
- **printstr (--) -** Print Name Of Entry Image.
  This will write a string to the console when the flash read command pointer is positioned to the fifth byte of a directory entry.
- **empty_entry (sector16k -- entry#) -** Return First Empty Entry Number.
  Scan the entries in a directory until the first empty entry is found. Return its entry number.
- **entry_addr (entry# sector16k -- double_rom_addr) -**Flash Address Of Entry.
  Given the 16KB location of a directory and a desired entry number, this will return the 24-bit flash address of the directory entry.
- **goto_entry (entry# sector16k --) -** Set Read Pointer To Entry.
  Given the 16KB location of a directory and a desired entry number, this will issue a flash Read command to position the Read command pointer to the start of the specified directory entry.
- **mark_reclaim (entry# sector16k --) -** Mark Entry For Deletion.
  Given the 16KB location of a directory and a desired entry number, this will mark the entry as Reclaimable. The entry and its associated image are marked for deletion by the **pack** command.

7.8.3 Directory Commands

The following commands generalize the operations on a directory. Normally the Forth directory image directory and the HP-71B ROM/IRAM image directory are specified using the `fthstart` and `romstart` constants defined in the *soft-spi.fs* file. Other directories can be created and manipulated if so desired.

- **dir_init (nblocks sector16k --) -** Initialize Directory.
  Erase the two blocks making up a directory. The `sector16k` value is either `fthstart` or `romstart`. An initial first entry is added indicating the number of image blocks reserved following the directory itself.
- **dir_size (sector16k -- nblocks) -** Return Directory Size.
  Return the number of image blocks allocated to the specified directory. The first directory entry is of type DIRSIZE and holds the number of allocated blocks, stored in the name field.

- **dir_find (name sector16k -- entry#) -** Find Named Directory Entry.
  Given a directory address `sector16k` and a directory image string name, return the `entry#` location in the directory. If the name is not found then an `entry#` value of -1 is returned.
- **dir_insert (name type.size sector16k -- block) -** Insert Directory Entry.
  Given a directory address `sector16k`, a directory image string name, and a `type.size` value defining the image associated with the entry, insert an entry in the directory and return the block where the image should be stored. The caller is responsible for insuring the name does not already exist.
- **dir_drop (name sector16k --) -** Mark Directory Entry Reclaimable.
  Given a directory address `sector16k` and a directory image string name, mark the directory entry as invalid and reclaimable.

- **dir_free (sector16k -- number) -** Available Directory Image Blocks.
  Return the number of remaining unused image blocks in the `sector16k` directory. A zero value indicates the directory needs to be packed.
- **dir_list (sector16k --) -** List directory entries.
  List the valid entries in a directory. The `sector16k` value is normally either `fthstart` or `romstart`.
- **dir_pack (sector16k --) -** Pack directory.
  The `sector16k` value is either `fthstart` or `romstart`. This command will pack both the directory and the image storage associated with the directory.

7.9 File Transfer Support

These Forth words support the transfer of ROM/IRAM images to and from the MultiMod II as well as transferring FPGA bitstream images to the MultiMod II flash storage. The latter capability is used to update the production configuration of the MultiMod II as well as to install optional configurations.

File transfer is done using the Kermit protocol, which is supported by most terminal emulators.

7.9.1 Forth Dictionary Images

Forth dictionary images can be loaded or saved via the Forth dictionary using the following commands. Note that Forth words in a text file can be simply sent via a terminal emulator to the Forth console rather than typing in definitions by hand.

- **forthsave (name --) -** Save Forth Dictionary Image.
  If `name` doesn't exist in the dictionary it is created, otherwise the current image updates the image stored in flash.
- **forthload (name --) -** Load Forth Dictionary By Name.
  The current Forth dictionary is replaced by the `name` image in flash. If that name doesn't exist in the dictionary, then no change occurs.
- **forthlist (--) -** List Forth Dictionary Entries.
  This uses the `dir_list` command from the `directory.fs` set of words to list entries in the Forth directory.

7.9.2 HP-71B ROM/IROM Images

ROM image `.bin` files can be sent using the Kermit protocol. ROM/IRAM images in flash can be downloaded and automatically given the `.bin` extension.

- **romverify (ram# name -- ram_addr flag) -** Verify Flash Image By Name.
  Given a name in the ROM directory and the starting point of an image in SPRAM, verify the content in SPRAM matches that of the image in SPRAM. The size of the image in the directory entry is used to determine the length of the image in 16KB sectors. The last memory address examined is returned along with a Pass/Fail flag. If the images don't match the **ram_addr** indicates the point of first mismatch.
- **writeflash (name type --) -** Upload ROM/IROM Image To Flash.
  Uploads a file and stores the file in flash, adding *name* to the ROM/IROM directory.
- **readflash (name --) -** Download Flash Image By Name.
  Downloads the *name* ROM/IROM image using the Kermit receive protocol.
- **romlist (--) -** List HP-71B Directory Entries.
  This uses the `dir_list` command from the `directory.fs` set of words to list ROM/IRAM image entries in the HP-71B directory.

7.9.3 FPGA Bitstream Images

There are four locations, slots 0 through 3, in the first megabyte of flash memory that store bitstream images. The first, slot 0, is the bootloader and cannot be erased or altered. The second, slot 1, is the MultiMod II production configuration that is loaded while the MultiMod II is in the HP-71B card reader cavity. Slots 2 and 3 are for alternate configurations that can be loaded via a warm boot.

- **bitstream (slot --) -** Transfer FPGA Bitstream.
  Uploads a bitstream file to flash and loads it as bitstream number `slot`. The slot number does not include 0, the bootloader bitstream image itself.